Laboratorio: Uso de Expresiones Regulares en Python

Teoría de Lenguajes

7 de abril de 2026

1 Introducción

El objetivo del presente laboratorio es que los estudiantes utilicen expresiones regulares para dar soluciones a problemas que se pueden encontrar en la práctica al procesar y/o trabajar con archivos o entradas de texto. Las expresiones regulares pasarán a ser una herramienta a tener en cuenta para utilizar en el tratamiento de textos, como archivos de texto en lenguaje natural, logs de programas y archivos de código, entre muchos otros. Podrá utilizarlas de forma sencilla en lenguajes de programación (ej. pearl, python, ruby, etc.), a través de comandos de terminal (ej. grep, sed, awk, etc.) o editores de texto. Por lo antes mencionado, se desaconseja el uso de herramientas de IA generativa para la realización de este trabajo. Entendemos que el aprendizaje y realización de este laboratorio es de gran utilidad y aporte a la formación del estudiante [5].

En este laboratorio se pide escribir un conjunto de rutinas en el lenguaje de programación Python 3. Se espera que los problemas sean resueltos con expresiones regulares, evitando utilizar estructuras de control (ej. for, while, if, etc.) u otras herramientas para resolverlos.

2 Modo de Trabajo

En esta letra se indicará un conjunto de programas a realizar. Para cada programa, se entregan archivos de entrada junto con sus correspondientes archivos de salida, los cuales corresponden al resultado esperado para cada caso. Cada grupo deberá implementar los programas pedidos considerando la especificación en esta letra, las entradas y sus correspondientes salidas.

- La salida del programa del grupo deberá ser exactamente igual a la salida de referencia. También se deberán mantener los nombres de los archivos de entrada y salida de los programas.
- Cada grupo debe entregar los programas implementados y el archivo "integrantes.txt" que se detalla más adelante en esta letra.

3 Realidad planteada

La empresa Teoleng S.A. necesita conciliar de forma automática las facturas de proveedores y confirmar que el crédito se haya ingresado en el banco para poder continuar con el flujo de operaciones. Para esto, se contará con un único archivo XML descargado directamente desde el banco y cinco archivos PDF asociados a facturas que se deben verificar estén correctamente en el estado bancario.

XML, o eXtensible Markup Language, es un lenguaje de marcado utilizado para almacenar datos de forma ordenada y estructurada. En esta tarea se trabajará utilizando como entrada un archivo XML que contiene los datos de las transacciones que afectan la cuenta bancaria de interés. El objetivo es utilizar expresiones regulares para extraer información contenida en él, simulando un escenario donde no se cuenta con herramientas especializadas para procesar XML. No se permitirá el uso de librerías que trabajen directamente con el XML.

El formato **PDF** (Portable Document Format) es un tipo de archivo diseñado para representar documentos de manera independiente del software, hardware o sistema operativo utilizado, preservando su apariencia visual original. Se trabajará con cinco archivos PDF que contendrán la información corre-

spondiente a las facturas (fechas, ítems facturados, cantidades, precios, etc.). El archivo PDF deberá ser leído como texto plano.

4 Programas a implementar

A continuación se presentan seis programas a implementar en Python, que deberán analizar el archivo **TeoLeng.xml** y los cinco documentos PDF, utilizando el módulo de expresiones regulares [2].

4.1 programa1.py

Recibe la ruta relativa de un archivo PDF como parámetro y devuelve su contenido como texto (string).

4.2 programa2.py

Dada una factura, despliega la fecha y el "débito bancario" con el siguiente formato:

```
Fecha: 2026-03-16 | Monto: 954,25
```

Sugerencia: Revisar el formato de las fechas y normalizar con el formato aaaa-mm-dd. Se permite el uso de la librería datetime [6].

4.3 programa3.py

Despliega todas las cantidades, la descripción, precio por unidad y precio total de los ítems facturados con el siguiente formato:

```
Cant: 1 |Desc: Item1 | 200,11 c/u |Total: 200,11
Cant: 1 |Desc: Item 2 | 0,70 c/u |Total: 0,70
```

4.4 programa4.py

Este programa se encarga de leer el archivo `xml` e imprimir su contenido como texto plano (string).

4.5 programa5.py

Dada una factura y el estado bancario, este programa devuelve `True` si hay un movimiento coincidente (misma fecha e importe que la factura), o `False` en otro caso.

Sugerencia: Usar el programa2 para obtener la fecha y el "débito bancario" que se deberá buscar en el XML.

4.6 programa6.py

Dada una factura y el estado bancario `xml`, el programa elimina la línea coincidente. En caso de que no exista coincidencia, no se realiza ninguna modificación.

Ejemplo: Con `factura1.pdf` (fecha = 2026-02-25 y débito bancario = 3210,92) y el siguiente `xml` como entrada:

```xml
<?xml version="1,0" encoding="utf-8"?>
<BanTeng:Estado xmlns:BanTeng="BanTeng-EstadoDeCuenta">
 <BanTeng:Saldo Cuenta="APELLIDO, NOMBRE" Moneda="UYU"
   TipoCuenta="12345678 - CC - 0 - \$" />
  <BanTeng:Movimientos>
   <BanTeng:Desde>2026-02-02</BanTeng:Desde>
   <BanTeng:Hasta>2026-03-31</BanTeng:Hasta>
    <BanTeng:Movimiento TipoMov="C" Importe="3210,92"
   Fecha="2026-02-25" Concepto="factura 1" Saldo="0" />
    <BanTeng:Movimiento TipoMov="C" Importe="1051,56"
   Fecha="2026-02-27" Concepto="Transferencia en plaza 1"
   Saldo="" />
   <BanTeng:TotalMovimientos>2</BanTeng:TotalMovimientos>
  </BanTeng:Movimientos>
</BanTeng:Estado>
```

La salida esperada es:

```xml
<?xml version="1,0" encoding="utf-8"?>
<BanTeng:Estado xmlns:BanTeng="BanTeng-EstadoDeCuenta">
 <BanTeng:Saldo Cuenta="APELLIDO, NOMBRE" Moneda="UYU"
   TipoCuenta="12345678 - CC - 0 - \$" />
  <BanTeng:Movimientos>
   <BanTeng:Desde>2026-02-02</BanTeng:Desde>
   <BanTeng:Hasta>2026-03-31</BanTeng:Hasta>
   <BanTeng:Movimiento TipoMov="C" Importe="1051,56" Fecha
   ="2026-02-27" Concepto="Transferencia en plaza 1" Saldo
   ="" />
   <BanTeng:TotalMovimientos>1</BanTeng:TotalMovimientos>
  </BanTeng:Movimientos>
</BanTeng:Estado>
```

Nota: Observar que se elimina por completo el elemento XML asociado al movimiento, manteniendo la estructura `xml`. Además, se actualiza el contenido del elemento `BanTeng:TotalMovimientos` con el número de movimientos restantes.

5 Entrega

La entrega se realizará a través de un formulario en EVA.
La fecha límite es el **24 de abril a las 23:59**.

Se deberá entregar:

- Los archivos `programa1,2,3,4,5,6.py` con la implementación de los programas
- Un archivo integrantes.txt con las cédulas (sin puntos ni dígito de verificación) y el nombre de los integrantes del grupo, uno por línea, separado por comas, como se muestra en el ejemplo. La primera línea debe contener el número de integrantes del grupo. Opcionalmente, se puede utilizar el final de este archivo para comentarios que el grupo considere pertinentes.

Ejemplo de archivo integrantes.txt:

```
3
7123456, ApellidoA1 ApellidoA2, NombreA1 NombreA2
8654321, ApellidoB1 ApellidoB2, NombreA1 NombreB2
9876543, ApellidoC1 ApellidoC2, NombreC1 NombreC2
Hicimos dos soluciones para el programa3, entregamos la que consideramos mejor y dejamos la otra comentada
```

Referencias

1. Python 3 Documentation
2. Módulo `re` de Python
3. Módulo `pypdf`
4. `Regex`
5. `Uso de IA`
6. Módulo `datetime` de Python